Glam Lavish — Inventory Management System

Product Requirements Document (PRD)

Document Version	1.0
Date	March 12, 2026
Status	Draft
Project	Glam Lavish Inventory & Order Management

---

1. Project Overview

1.1 Background

Glam Lavish is an e-commerce business that currently operates through a WooCommerce website. As order volume grows, there is a need for a dedicated **inventory management system** that works alongside the existing WooCommerce store — providing centralized product management, streamlined order processing, automated courier dispatching, and real-time stock synchronization.

1.2 Goals

- **Centralized Inventory Control** — Manage all products, stock levels, and variations from a single admin panel
- **Bidirectional WooCommerce Sync** — Orders and stock changes flow seamlessly between the inventory system and WooCommerce
- **Automated Order Fulfillment** — Orders automatically dispatch to Steadfast courier upon creation
- **Invoice Generation** — Print thermal receipts (3×4 inch) for each order with QR-based tracking

- **Real-time Order Tracking** — Customers can scan a QR code to track their order status

1.3 Success Criteria

- Orders created in the inventory system automatically appear with a Steadfast consignment ID
- WooCommerce orders appear in the inventory system within seconds (via webhooks)
- Stock levels stay synchronized across both platforms
- Invoices print correctly on a thermal printer
- QR code scan opens a tracking page with accurate status

---

2. User Personas

2.1 Admin (Business Owner)

- Full access to all features
- Manages products, categories, and staff accounts
- Views dashboard with business metrics
- Configures WooCommerce and courier API settings

2.2 Staff (Order Processor)

- Creates and manages orders
- Prints invoices
- Views product stock levels
- Cannot manage users or system settings

2.3 Customer (End User)

- Does not log in to the inventory system
- Scans QR code on invoice to track order status via a public tracking page

---

3. Tech Stack

Layer	Technology
Backend	NestJS (TypeScript)
Frontend / Admin Panel	React (TypeScript) + Vite + Tailwind CSS
Database	PostgreSQL + TypeORM
Authentication	JWT (email/password)
Courier Integration	Steadfast API, Pathao API
WooCommerce Integration	WooCommerce REST API v3

---

4. Functional Requirements

4.1 Product Module

FR-4.1.1 Product Management

Field	Type	Required	Notes
Product Name	Text	Yes	
Short Description	Text	No	Brief summary
Description	Rich Text	No	Full product description
Product Image	File Upload	No	Primary product image
Category	Select	No	Links to Category entity
Product SKU	Text	Yes	Unique identifier
Product Type	Enum	Yes	SIMPLE or VARIABLE

FR-4.1.2 Simple Product Fields

Field	Type	Required
Stock Quantity	Integer	Yes
Regular Price	Decimal (BDT)	Yes
Sale Price	Decimal (BDT)	No

FR-4.1.3 Variable Product — Variations

Each variation of a variable product contains:

Field	Type	Required
Color	Text	No
Size	Text	No
Variation SKU	Text	Yes (unique)
Regular Price	Decimal (BDT)	Yes
Sale Price	Decimal (BDT)	No
Stock Quantity	Integer	Yes
Variation Image	File Upload	No

FR-4.1.4 Product Sync with WooCommerce

- Products created/updated locally are pushed to WooCommerce
- Products created/updated in WooCommerce sync to the local system via webhooks
- Each product/variation stores a `wcId` to link with WooCommerce

---

4.2 Category Module

Feature	Description
Create Category	Name, slug, optional WooCommerce ID link
List Categories	Filterable list of all categories
Edit/Delete	Standard CRUD operations
WC Sync	Categories sync bidirectionally with WooCommerce

4.3 Invoice / Order Generator

FR-4.3.1 Order Creation

Field	Type	Required	Notes
Invoice ID	Auto-generated	Yes	Format: `GL-XXXX` (sequential)
Products	Multi-select	Yes	Select product + variation + quantity
Customer Name	Text	Yes	
Customer Phone	Text	Yes	Bangladesh mobile format
Customer Address	Text	Yes	Full delivery address
Shipping Partner	Enum	Yes	Steadfast / Pathao
Shipping Zone	Enum	Yes	Determines shipping fee

FR-4.3.2 Shipping Zones & Fees

Zone	Fee (BDT)
Inside Dhaka	80
Dhaka Sub Area	100
Outside Dhaka	150

FR-4.3.3 Order Creation Flow

1. Staff selects products and quantities (stock validated in real-time)
2. Enters customer details and selects shipping zone
3. System auto-calculates: `Subtotal + Shipping Fee = Grand Total`
4. On submit:
    - Invoice ID generated atomically (GL-0001, GL-0002, ...)
    - Stock decremented for each product/variation
    - **Order automatically pushed to Steadfast courier API**
    - QR code generated linking to public tracking page
    - Stock synced to WooCommerce
5. Order confirmation shown with invoice ID, courier tracking ID, and QR code

FR-4.3.4 Order Statuses

Status	Description
PENDING	Order created, awaiting confirmation
CONFIRMED	Order confirmed and being prepared
PROCESSING	Order being packed/processed
SHIPPED	Handed to courier
DELIVERED	Delivered to customer
CANCELLED	Order cancelled
RETURNED	Order returned by customer

FR-4.3.5 Invoice Print (3×4 Inch Thermal Receipt)

The invoice prints on a 3-inch × 4-inch thermal receipt with the following layout:

```
┌──────────────────────────┐
│      Glam Lavish         │
```

```
│                          │
│ Invoice: GL-0001         │
│ Date: 12/03/2026         │
│ Courier: Steadfast       │
│ Tracking: 227241927      │
├──────────────────────────┤
│ To: Customer Name        │
│ Ph: 01XXXXXXXXX          │
│ Addr: Full address       │
├──────────────────────────┤
│ Product    Qty    Price  │
│ Item 1      1   1,499.00│
├──────────────────────────┤
│ Subtotal:       1,499.00│
│ Delivery:          80.00│
│ Grand Total:   1,579.00 │
│ Due Amount:    1,579.00  │
│                          │
│        [QR Code]         │
└──────────────────────────┘
```

- Uses CSS `@page { size: 3in 4in; margin: 2mm; }` for thermal printing
- `react-to-print` library triggers the browser print dialog
- QR code (~2cm × 2cm) at the bottom encodes the tracking URL

4.4 QR Code & Order Tracking

FR-4.4.1 QR Code Generation

- Generated at order creation time using the `qrcode` npm package
- Encodes URL: `{FRONTEND_URL}/tracking/{invoiceId}`
- Stored as a data URL on the Order record
- Rendered on the invoice and in the order detail page

FR-4.4.2 Public Tracking Page

- **No authentication required** — accessible by anyone with the URL/QR code
- Displays:
  - Order status timeline (visual progress: Pending → Confirmed → Shipped → Delivered)
  - Invoice ID and order date
  - Customer name (partially masked for privacy)
  - Courier name and tracking ID
  - Link to courier's own tracking page (if available)

4.5 WooCommerce Integration

FR-4.5.1 Configuration

Setting	Description
WooCommerce Store URL	The WordPress/WooCommerce site URL
Consumer Key	WC REST API consumer key (Read/Write)
Consumer Secret	WC REST API consumer secret
Webhook Secret	For verifying incoming webhook signatures

> **Setup Note:** Generate API keys from WooCommerce → Settings → Advanced → REST API → Add Key. Select "Read/Write" permissions.

FR-4.5.2 Inbound Sync (WooCommerce → Inventory)

Webhook Event	Action
`order.created`	Create local Order + OrderItems, decrement stock
`order.updated`	Update local order status and details
`product.created`	Create local Product with variations
`product.updated`	Update local product details and stock

- Webhooks verified via `X-WC-Webhook-Signature` (HMAC-SHA256)
- Matched to local records via `wcId` / `wcOrderId`

FR-4.5.3 Outbound Sync (Inventory → WooCommerce)

Event	API Call
Local order created	`PUT /wc/v3/products/{id}` — update stock quantity
Local product created	`POST /wc/v3/products` — create product in WC
Local product updated	`PUT /wc/v3/products/{id}` — update product in WC
Variable product stock change	`PUT /wc/v3/products/{id}/variations/{vid}`

FR-4.5.4 Deduplication

- When outbound sync triggers a WC webhook back, the system checks `wcLastSyncedAt` timestamp
- If the entity was synced within the last 5 seconds, the inbound webhook is skipped
- All sync operations logged in `SyncLog` table

FR-4.5.5 Manual Sync

- "Sync Now" button in Settings triggers a full sync:
  - Fetches all WC products (paginated, 100/page) and upserts locally
  - Fetches recent WC orders (last 30 days) and upserts locally
- Sync progress and results displayed in a sync log viewer

4.6 Courier Integration — Steadfast

FR-4.6.1 Auto-Push on Order Create

When an order is created in the inventory system, it is **automatically** pushed to Steadfast:

Detail	Value
API Endpoint	`POST https://portal.steadfast.com.bd/api/v1/create_order`
Auth Headers	`Api-Key` , `Secret-Key`
Payload	`invoice` , `recipient_name` , `recipient_phone` , `recipient_address` , `cod_amount`
Response	Returns `consignment_id` and `tracking_code`

- `consignment_id` and `tracking_code` stored on the Order record
- If Steadfast API fails, the order is still created but flagged with `courierConsignmentId = null`
- A "Retry Courier" button appears in the order detail for manual retry

FR-4.6.2 Status Polling

- A scheduled cron job runs every 15 minutes
- Queries all orders with status `SHIPPED` and a valid `courierConsignmentId`
- Calls `GET /api/v1/status_by_cid/{consignment_id}`

- Maps Steadfast delivery status to local `OrderStatus` enum
- Updates order status automatically

FR-4.6.3 Pathao Courier (Future)

- OAuth2 token-based authentication
- Order creation via `POST /aladdin/api/v1/orders`
- Similar status polling mechanism
- Implemented in Phase 6

4.7 Dashboard

Widget	Description
Total Orders Today	Count of orders created today
Revenue Today	Sum of `grandTotal` for today's orders
Pending Orders	Count of orders in PENDING status
Low Stock Alerts	Products with stock below configurable threshold
Recent Orders	Table showing last 10 orders with status badges

---

5. Database Schema Overview

5.1 Entity Relationship Summary

```
User ──< Order ──< OrderItem >── Product
                                  │
                    ProductVariation ┘
                          │
Category ──< Product      │
                          │
Order ──< SyncLog         │
                          │
InvoiceCounter (singleton)
```

5.2 Key Entities

Entity	Key Fields	Notes
User	id, email, password, name, role	Roles: ADMIN, STAFF
Category	id, name, slug, wcId	WC-linked

Product	id, name, sku, type, prices, stockQuantity, wcId	Soft delete
ProductVariation	id, productId, sku, color, size, prices, stock, wcId	Belongs to variable product
Order	id, invoiceId, source, status, customer info, totals, courier info, wcOrderId	Central entity
OrderItem	id, orderId, productId, variationId, quantity, prices	Line items
SyncLog	id, direction, entityType, status, payload, error	Audit trail
InvoiceCounter	id (singleton), lastNum	Atomic ID generation

5.3 TypeORM Implementation Notes

- All entities use UUID primary keys
- Money fields: `decimal` type with `precision: 10, scale: 2`
- Enums: stored as PostgreSQL enum types
- Soft delete on Product via `@DeleteDateColumn()`
- Invoice counter uses `QueryRunner` with `SELECT ... FOR UPDATE` row locking
- Timestamps: `createdAt` and `updatedAt` on all entities via `@CreateDateColumn()` / `@UpdateDateColumn()`

---

6. API Endpoints Overview

6.1 Authentication

Method	Endpoint	Description
POST	`/api/auth/login`	Login with email/password, returns JWT
GET	`/api/auth/me`	Get current authenticated user

6.2 Products

Method	Endpoint	Description
GET	`/api/products`	List products (paginated, filterable)
GET	`/api/products/:id`	Product detail with variations
POST	`/api/products`	Create product (simple or variable)
PATCH	`/api/products/:id`	Update product
DELETE	`/api/products/:id`	Soft delete product
POST	`/api/products/:id/variations`	Add variation
PATCH	`/api/products/variations/:id`	Update variation

6.3 Categories

Method	Endpoint	Description
GET	`/api/categories`	List all categories
POST	`/api/categories`	Create category
PATCH	`/api/categories/:id`	Update category
DELETE	`/api/categories/:id`	Delete category

6.4 Orders

Method	Endpoint	Description
GET	`/api/orders`	List orders (filterable)
GET	`/api/orders/:id`	Order detail with items
POST	`/api/orders`	Create order (auto Steadfast + WC sync)
PATCH	`/api/orders/:id/status`	Update order status

GET	`/api/orders/:id/invoice`	Invoice print data
GET	`/api/orders/:id/qr`	QR code image

6.5 Public Tracking

Method	Endpoint	Description
GET	`/api/tracking/:invoiceId`	Public order tracking (no auth)

6.6 WooCommerce

Method	Endpoint	Description
POST	`/api/woocommerce/webhook/order`	Receive WC order webhook
POST	`/api/woocommerce/webhook/product`	Receive WC product webhook
POST	`/api/woocommerce/sync/products`	Manual full product sync
POST	`/api/woocommerce/sync/orders`	Manual full order sync
GET	`/api/woocommerce/sync-logs`	View sync history

6.7 Dashboard

Method	Endpoint	Description
GET	`/api/dashboard/stats`	Summary statistics
GET	`/api/dashboard/low-stock`	Low stock product alerts
GET	`/api/dashboard/recent-orders`	Last 10 orders

7. Frontend Pages

Route	Page	Auth	Description
`/login`	Login	Public	Email/password login form
`/`	Dashboard	Protected	Stats, recent orders, low-stock alerts
`/products`	Product List	Protected	Searchable, filterable product table
`/products/new`	Product Form	Protected	Create simple/variable product
`/products/:id/edit`	Product Form	Protected	Edit existing product
`/orders`	Order List	Protected	Filterable by status, source, date
`/orders/new`	Order Form	Protected	**Main order generator** with product search, line items, customer info, shipping calculator
`/orders/:id`	Order Detail	Protected	View order, update status, courier info, retry courier
`/orders/:id/invoice`	Invoice Print	Protected	3×4 thermal receipt, print button
`/tracking/:invoiceId`	Tracking	**Public**	Order status timeline, courier tracking
`/settings`	Settings	Admin only	WC API config, user management

8. Non-Functional Requirements

8.1 Performance

- Order creation (including Steadfast push) completes within 3 seconds
- Product listing loads within 1 second for up to 1,000 products
- WooCommerce webhook processing completes within 2 seconds

8.2 Security

- All API endpoints (except tracking and WC webhooks) require JWT authentication
- Passwords hashed with bcrypt (minimum 10 salt rounds)
- WooCommerce webhooks verified via HMAC-SHA256 signature
- Steadfast/Pathao API keys stored as environment variables, never in code
- Input validation on all endpoints using `class-validator`

8.3 Reliability

- Order creation uses database transactions — if any step fails, everything rolls back
- Courier API failures do not block order creation — orders are flagged for retry
- WooCommerce sync failures are logged and can be retried manually
- Invoice counter uses row-level locking to prevent duplicate IDs under concurrent requests

8.4 Scalability

- Designed for a single-store operation (Glam Lavish)
- Can handle up to 500 orders/day comfortably
- Database indexes on frequently queried fields (invoiceId, wcId, status, createdAt)

---

9. Delivery Timeline

Phase	Duration	Deliverables
Phase 1: Scaffolding	Day 1–2	Project setup, database schema, auth module
Phase 2: Products	Day 3–5	Product CRUD, categories, product pages

Phase 3: Orders + Steadfast	Day 6–9	Order creation, auto Steadfast push, QR code, order pages
Phase 4: Invoice + Tracking	Day 10–11	Thermal invoice print, public tracking page
Phase 5: WooCommerce Sync	Day 12–15	Bidirectional sync, webhooks, manual sync, sync logs
Phase 6: Dashboard + Polish	Day 16–18	Dashboard, Pathao integration, cron jobs, UX polish

---

10. Risks & Mitigations

Risk	Impact	Mitigation
Steadfast API downtime	Orders created without courier dispatch	Retry mechanism + manual dispatch button
WooCommerce webhook failures	Stock desync between platforms	Manual "Sync Now" button + sync log monitoring
Concurrent invoice ID generation	Duplicate invoice numbers	Row-level locking with `SELECT ... FOR UPDATE`
WC sync loops (outbound triggers inbound webhook)	Infinite sync cycle	5-second deduplication window using `wcLastSyncedAt`

---

11. Future Enhancements

- Pathao courier full integration (Phase 6)
- Bulk order import/export (CSV)
- Product barcode scanning
- Customer database with order history
- SMS notifications for order status updates

- Multi-store support
- Financial reports and analytics
- Mobile app for order management

This document serves as the primary reference for the Glam Lavish Inventory Management System development. Requirements may evolve based on use-case feedback during development.